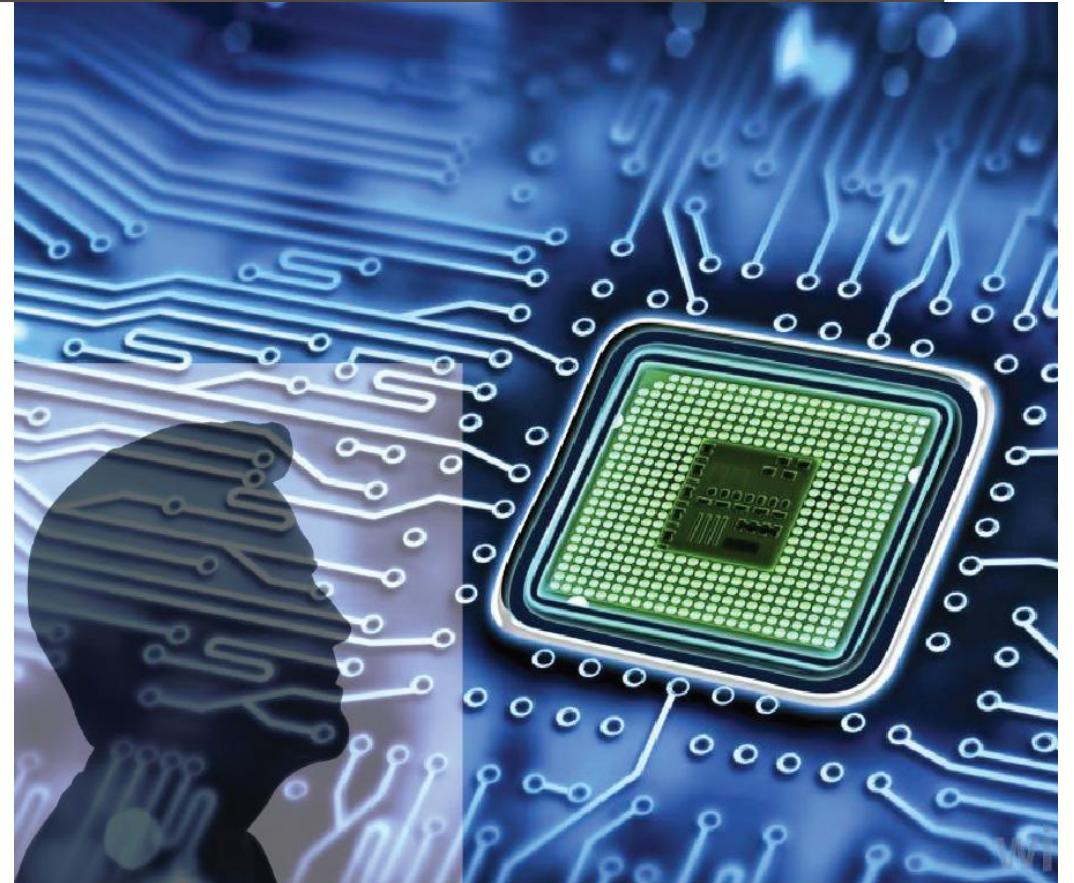
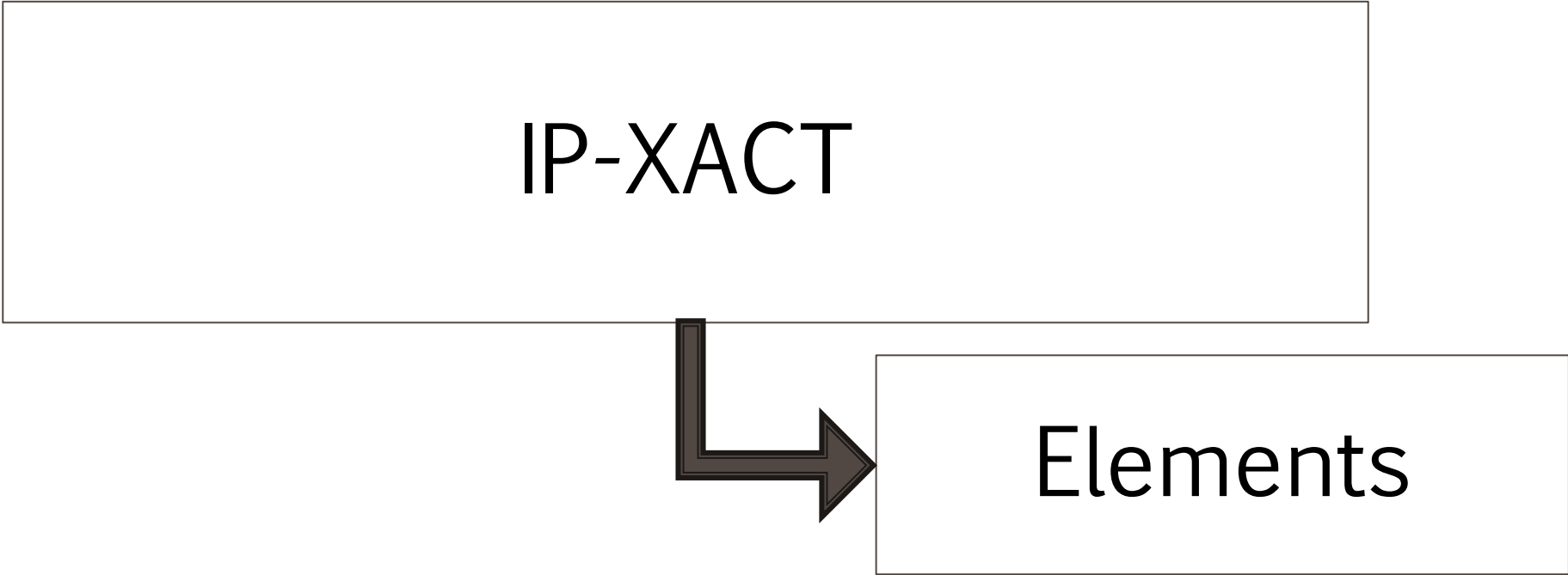


ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-5 Lecture-2 SoC Design Methodology

Sneh Saurabh
11th September, 2025



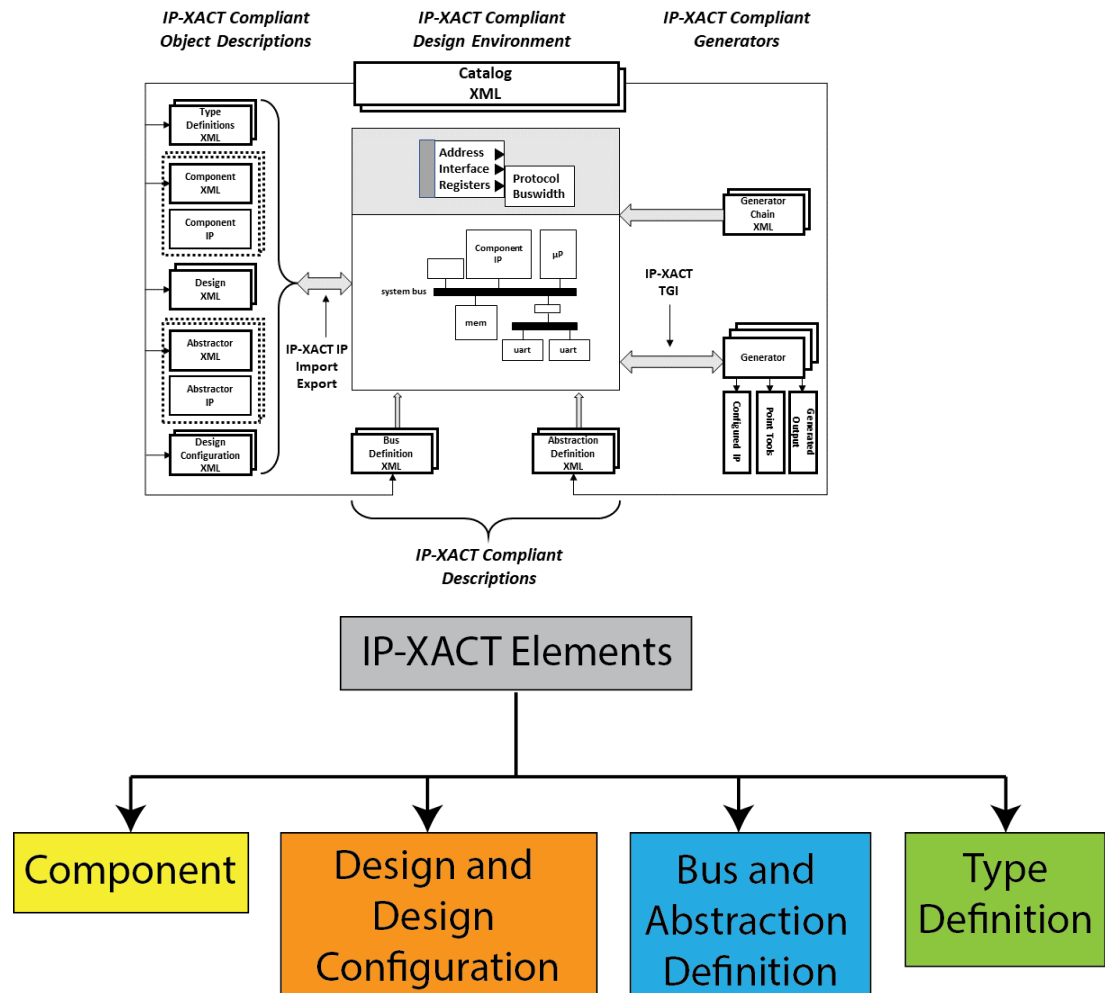


```
graph TD; A[IP-XACT] --> B[Elements]
```

IP-XACT

Elements

IP-XACT: Overview of Elements (1)

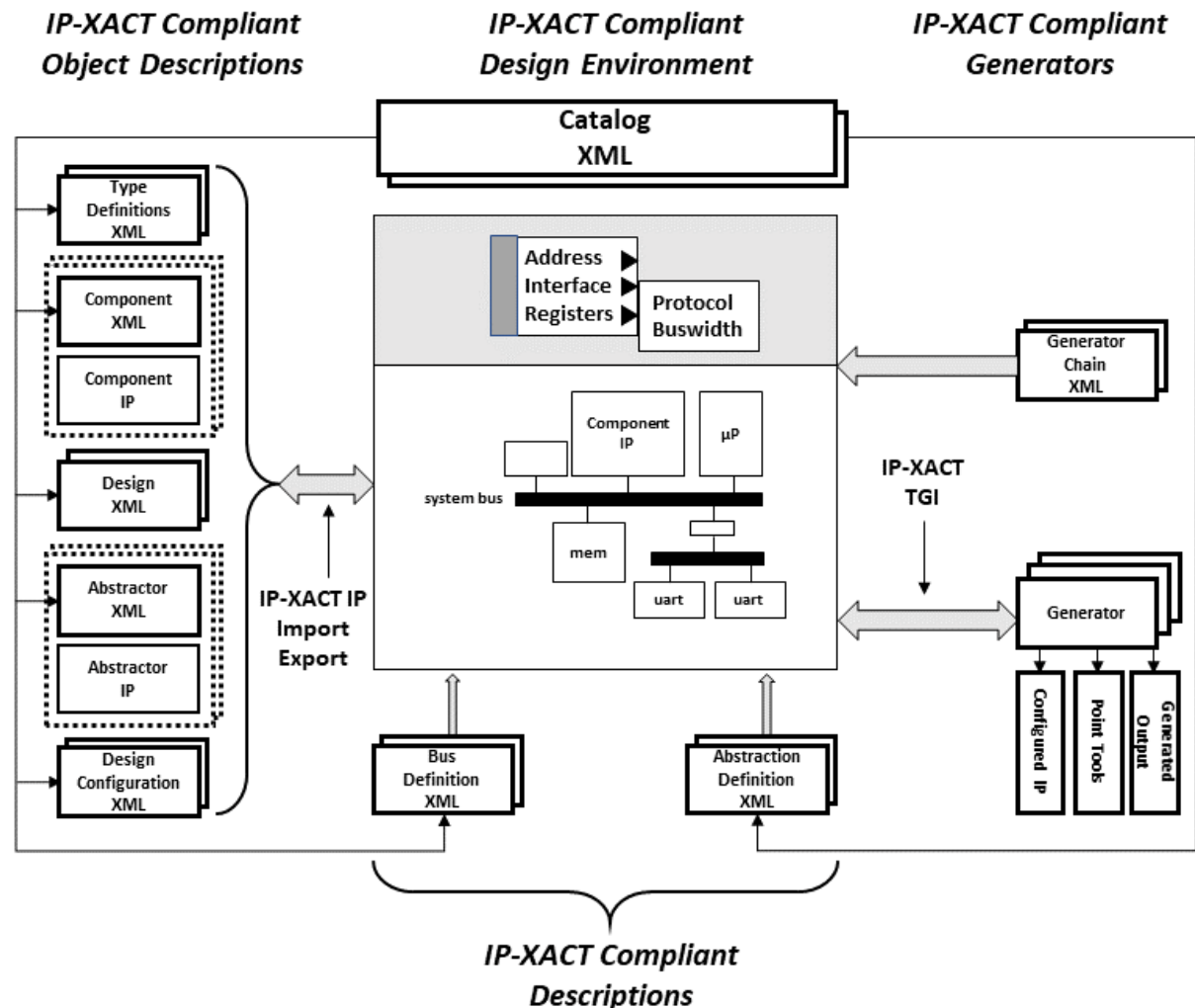


- **Component:** abstract representation of an IP, such as ports, parameters, registers, etc.
- **Design:** describe the structure of a hierarchical IP (instances of sub-components)
 - Design configuration: select various combination of views for sub-components and specifies values for parameters
- **Bus and Abstraction Definition:** define interface types for hardware communication protocols
- **Type Definition:** create definitions once and reuse their instances wherever required

IP-XACT: Overview of Elements (2)

Other elements

- **Abstractor:** IP blocks (similar to component) that implement conversion between two different abstraction definitions
- **Generator chain:** sequence of **generators** to describe flows enabled
 - **Generator:** a software tool (such as a script or an executable) that implements tasks for creating or manipulating information described in IP-XACT files
- **Catalog:** describe location and attributes of other IP-XACT top-level elements for managing collections of IP-XACT files efficiently



IP-XACT

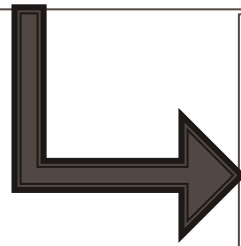
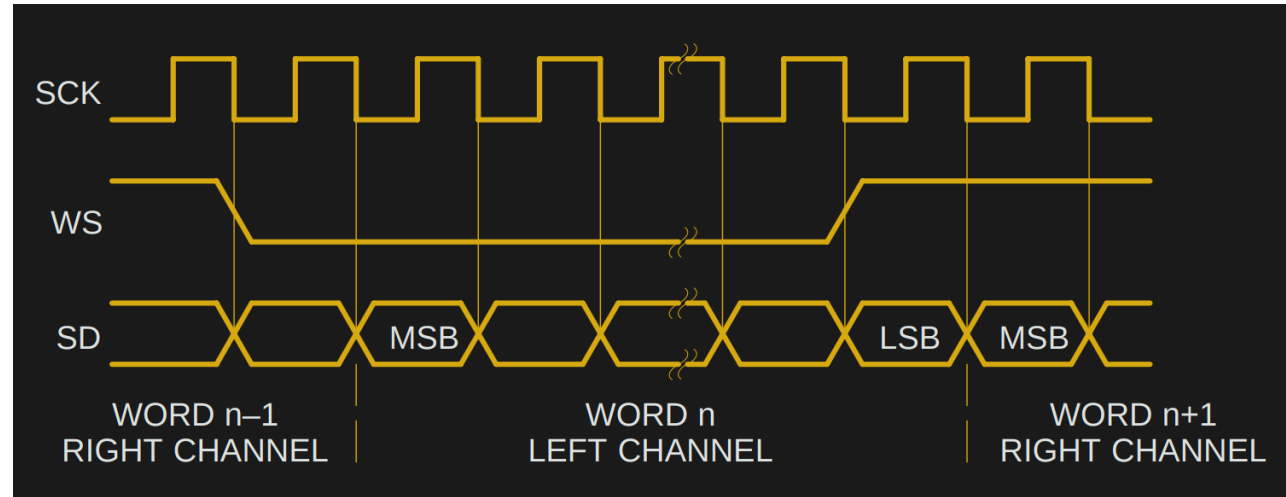


Illustration of IP-XACT
Elements

Illustration: I2S Protocol implementation

I2S protocol:

- Inter-IC Sound (I2S) protocol
- Used to transmit digital **audio** data from one device to another device
- Consists of three signals named SCK (serial clock), WS (word select), and SD (serial data)
- SD transmits payload serially
- WS separates the right and left channels



<https://hackaday.com/wp-content/uploads/2019/04/i2s-timing-themed.png>

- Initiator and Target participate in I2S communication
- The component that generates the SCK and WS signals is an initiator
- The component that receives the SCK and WS signals is a target

IP-XACT: Component RTL views (1)

- Transmitter can be either initiator or target



initiator_transmitter

```
module initiator_transmitter(sck, ws, sd);  
    output wire sck;  
    output wire ws;  
    output wire sd;  
endmodule
```

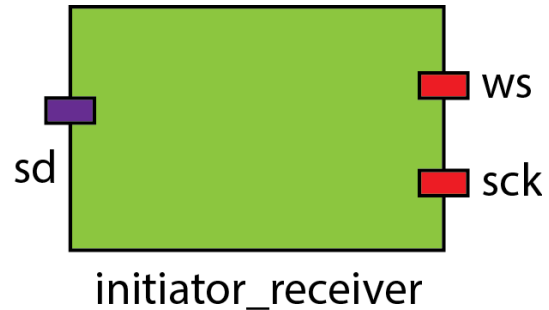


target_transmitter

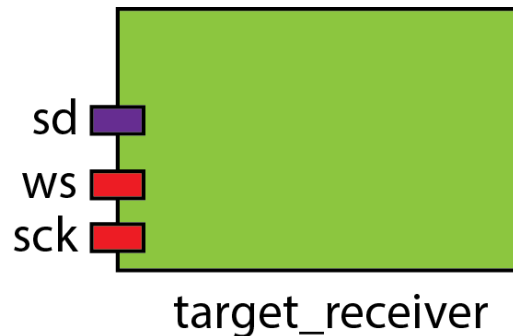
```
module target_transmitter(sck, ws, sd);  
    input wire sck;  
    input wire ws;  
    output wire sd;  
endmodule
```

IP-XACT: Component RTL views (2)

- Receiver can be either initiator or target



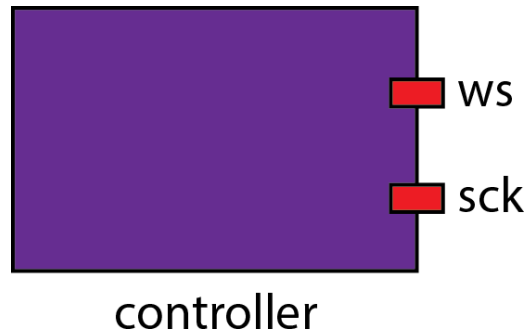
```
module initiator_receiver(sck, ws, sd);  
    output wire sck;  
    output wire ws;  
    input wire sd;  
endmodule
```



```
module target_receiver(sck, ws, sd);  
    input wire sck;  
    input wire ws;  
    input wire sd;  
endmodule
```

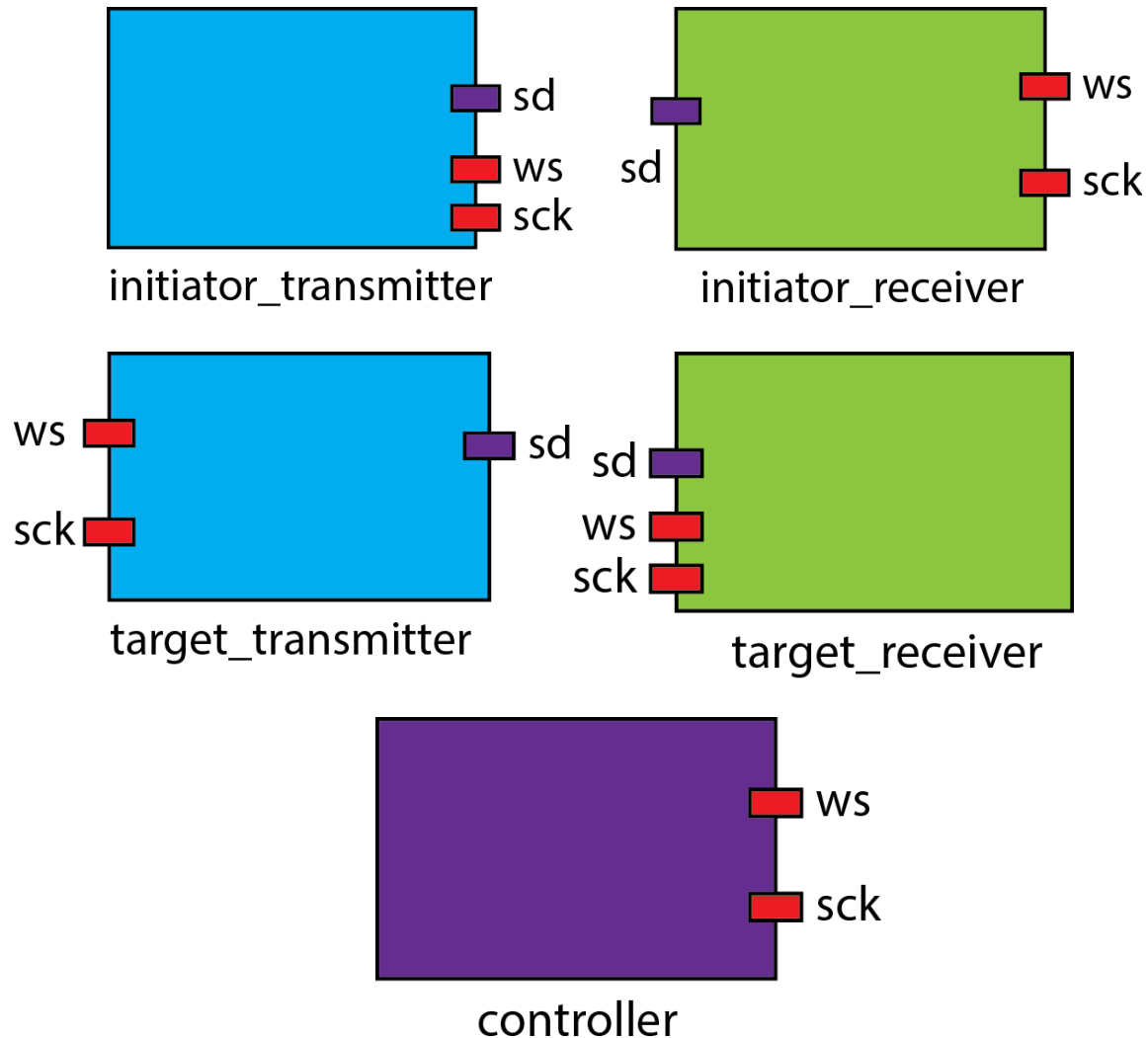
IP-XACT: Component RTL views (3)

- Separate controller can be the initiator



```
module controller(sck, ws);  
    output wire sck;  
    output wire ws;  
endmodule
```

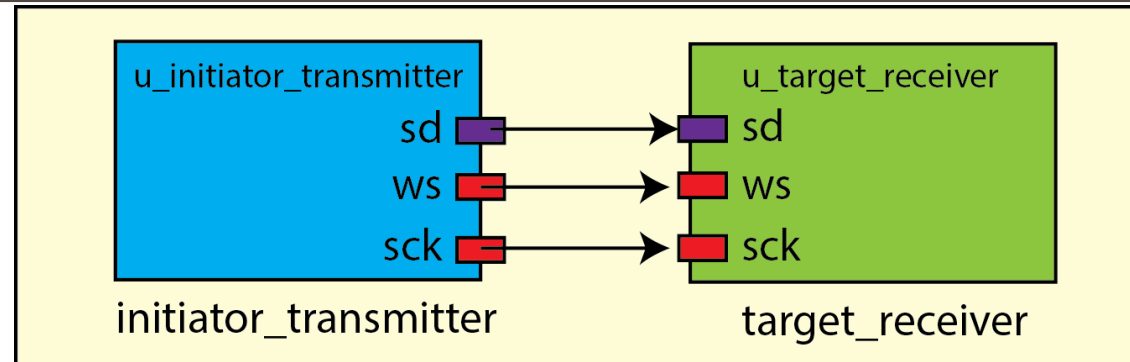
IP-XACT: Components



- IP-XACT **component** created corresponding to these IPs
- Each Verilog module is described as a single **view** in a single **component**.
 - Example: IP-XACT **component** initiator_transmitter contains a **view** describing the location of the Verilog file initiator_transmitter.v (details discussed later)
- The IP-XACT component also describes the ports with their names and directions.

IP-XACT: RTL view of structure (1)

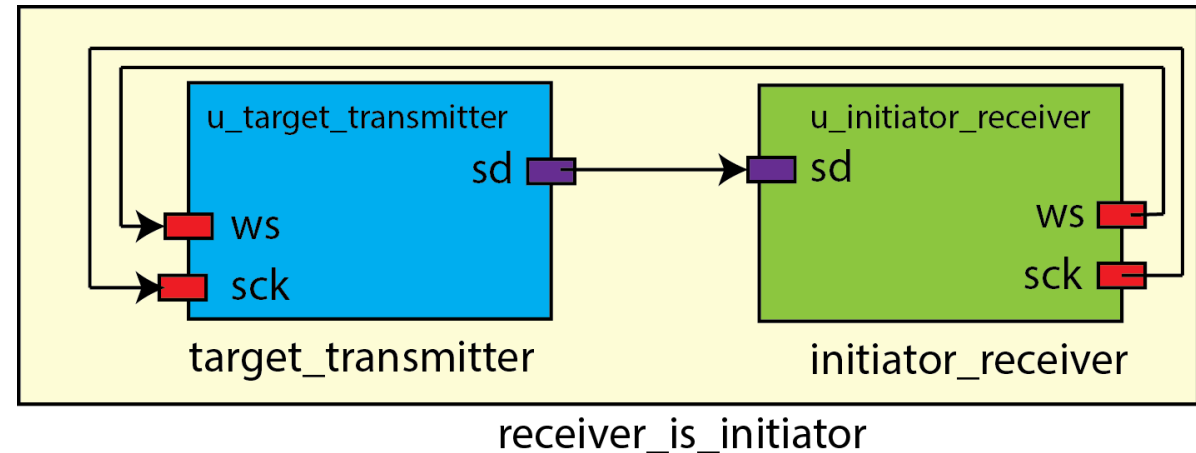
- The components can be connected in different ways
- Transmitter is the initiator



```
module transmitter_is_initiator;  
    wire u_initiator_transmitter_sck_sig;  
    wire u_initiator_transmitter_ws_sig;  
    wire u_initiator_transmitter_sd_sig;  
  
    initiator_transmitter u_initiator_transmitter ( .sck( u_initiator_transmitter_sck_sig )  
    , .ws( u_initiator_transmitter_ws_sig ), .sd( u_initiator_transmitter_sd_sig ) );  
  
    target_receiver u_target_receiver ( .sck( u_initiator_transmitter_sck_sig ), .ws( u_ini  
tiator_transmitter_ws_sig ), .sd( u_initiator_transmitter_sd_sig ) );  
  
endmodule
```

IP-XACT: RTL view of structure (2)

- Receiver is the initiator



```
module receiver_is_initiator;
  wire u_initiator_receiver_sck_sig;
  wire u_initiator_receiver_ws_sig;
  wire u_target_transmitter_sd_sig;

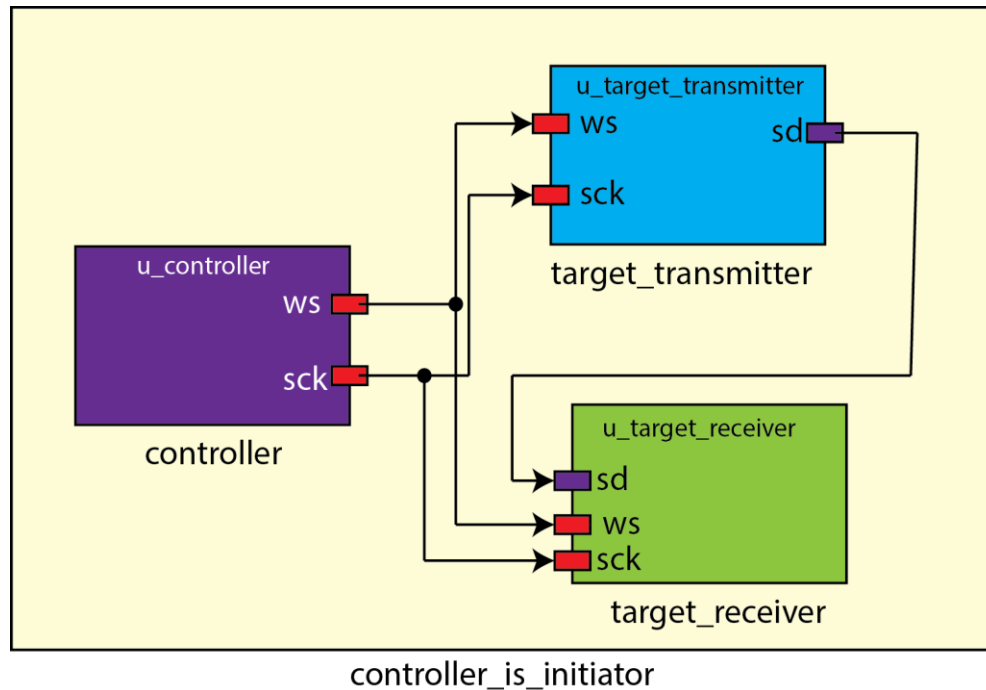
  initiator_receiver u_initiator_receiver ( .sck( u_initiator_receiver_sck_sig ),
    .ws( u_initiator_receiver_ws_sig ), .sd( u_target_transmitter_sd_sig ) );

  target_transmitter u_target_transmitter ( .sck( u_initiator_receiver_sck_sig ),
    .ws( u_initiator_receiver_ws_sig ), .sd( u_target_transmitter_sd_sig ) );

endmodule
```

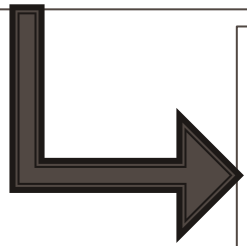
IP-XACT: RTL view of structure (2)

- Controller is the initiator



```
module controller_is_initiator;  
    wire u_controller_sck_sig;  
    wire u_controller_ws_sig;  
    wire u_target_transmitter_sd_sig;  
  
    controller u_controller (  
        .sck( u_controller_sck_sig ),  
        .ws( u_controller_ws_sig ) );  
  
    target_transmitter u_target_transmitter (  
        .sck( u_controller_sck_sig ),  
        .ws( u_controller_ws_sig ),  
        .sd( u_target_transmitter_sd_sig ) );  
  
    target_receiver u_target_receiver (  
        .sck( u_controller_sck_sig ),  
        .ws( u_controller_ws_sig ),  
        .sd( u_target_transmitter_sd_sig ) );  
  
endmodule
```

IP-XACT



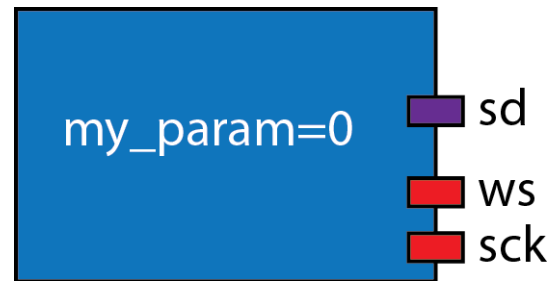
Details of Components
and Design

IP-XACT: Component

- A component documents one or more implementations of an IP (view)
- Different implementations are available in different files
 - Location of these files is documented in different file sets
- Component ports are described once
 - Ports can be tailored to specific views if needed, for instance to add language-specific type information

IP-XACT: Component

```
<ipxact:component xsi:schemaLocation="http://www.accellera.org/XMLSchema/IPXACT/1685-2022 http://www.accellera.org/XMLSchema/IPXACT/1685-2022/index.xsd">
  <ipxact:vendor>accellera.org</ipxact:vendor>
  <ipxact:library>12s</ipxact:library>
  <ipxact:name>initiator_transmitter</ipxact:name>
  <ipxact:version>1.0</ipxact:version>
  <ipxact:model>
    <ipxact:views>
      <ipxact:view>
        <ipxact:name>interface</ipxact:name>
        <ipxact:componentInstantiationRef>hdl-interface</ipxact:componentInstantiationRef>
      </ipxact:view>
    </ipxact:views>
    <ipxact:instantiations>
      <ipxact:componentInstantiation>
        <ipxact:name>hdl-interface</ipxact:name>
        <ipxact:language>verilog</ipxact:language>
        <ipxact:libraryName>initiator_transmitterlib</ipxact:libraryName>
        <ipxact:moduleName>initiator_transmitter</ipxact:moduleName>
        <ipxact:moduleParameters>
          <ipxact:moduleParameter parameterId="my_param" resolve="user" type="longint">
            <ipxact:name>my_param</ipxact:name>
            <ipxact:value>0</ipxact:value>
          </ipxact:moduleParameter>
        </ipxact:moduleParameters>
        <ipxact:fileSetRef></ipxact:fileSetRef>
      </ipxact:componentInstantiation>
    </ipxact:instantiations>
    <ipxact:ports>
      <ipxact:port>
        <ipxact:name>sck</ipxact:name>
        <ipxact:wire>
          <ipxact:direction>out</ipxact:direction>
        </ipxact:wire>
      </ipxact:port>
      <ipxact:port>
        <ipxact:name>ws</ipxact:name>
        <ipxact:wire>
          <ipxact:direction>out</ipxact:direction>
        </ipxact:wire>
      </ipxact:port>
      <ipxact:port>
        <ipxact:name></ipxact:name>
      </ipxact:port>
    </ipxact:ports>
  </ipxact:model>
  <ipxact:fileSets>
    <ipxact:fileSet>
      <ipxact:name>fs-interface</ipxact:name>
      <ipxact:file>
        <ipxact:name>../INTERFACE/initiator_transmitter.v</ipxact:name>
        <ipxact:fileType>verilogSource</ipxact:fileType>
        <ipxact:logicalName>initiator_transmitterlib</ipxact:logicalName>
      </ipxact:file>
    </ipxact:fileSet>
  </ipxact:fileSets>
</ipxact:component>
```



initiator_transmitter

```
module initiator_transmitter(sck, ws, sd);
  output wire sck;
  output wire ws;
  output reg sd;

  parameter my_param = 0;

endmodule
```

IP-XACT: Component

```
<ipxact:component xsi:schemaLocation="http://www.accellera.org/XMLSchema/IPXACT/1685-2022 http://www.accellera.org/XMLSchema/IPXACT/1685-2022/index.xsd">
  <ipxact:vendor>accellera.org</ipxact:vendor>
  <ipxact:library>i2s</ipxact:library>
  <ipxact:name>initiator_transmitter</ipxact:name>
  <ipxact:version>1.0</ipxact:version>
  <ipxact:model>
    <ipxact:views>
      <ipxact:view>
        <ipxact:name>interface</ipxact:name>
        <ipxact:componentInstantiationRef>hdl-interface</ipxact:componentInstantiationRef>
      </ipxact:view>
    </ipxact:views>
    <ipxact:instantiations>
      <ipxact:componentInstantiation>
        <ipxact:name>hdl-interface</ipxact:name>
        <ipxact:language>verilog</ipxact:language>
        <ipxact:libraryName>initiator_transmitterlib</ipxact:libraryName>
        <ipxact:moduleName>initiator_transmitter</ipxact:moduleName>
        <ipxact:moduleParameters>
          <ipxact:moduleParameter parameterId="my_param" resolve="user" type="longint"/>
        </ipxact:moduleParameters>
        <ipxact:fileSetRef/>
      </ipxact:componentInstantiation>
    </ipxact:instantiations>
  </ipxact:model>
</ipxact:component>
```

- Type of element is component
- Schema of component points to IP-XACT standard schema
- VLNV is specified for each top-level IP-XACT objects: **vendor**, **library**, **name**, **version**

- Multiple instantiations can be grouped together in a **view**
- The container element **views** can have multiple view elements.
- Each view has a **name** to identify the element.

- **componentInstantitation** describes the information that is needed to use the Verilog module declaration of this component (referred to in **view**):
 - Language, name, parameters, fileset

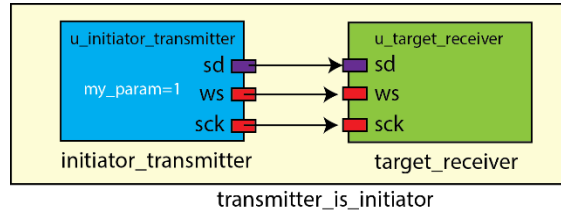
IP-XACT: Component

```
-<ipxact:ports>
  -<ipxact:port>
    <ipxact:name>sck</ipxact:name>
    -<ipxact:wire>
      <ipxact:direction>out</ipxact:direction>
    </ipxact:wire>
  </ipxact:port>
  -<ipxact:port>
    <ipxact:name>ws</ipxact:name>
    -<ipxact:wire>
      <ipxact:direction>out</ipxact:direction>
    </ipxact:wire>
  </ipxact:port>
  +<ipxact:port></ipxact:port>
</ipxact:ports>
</ipxact:model>
-<ipxact:fileSets>
  -<ipxact:fileSet>
    <ipxact:name>fs-interface</ipxact:name>
    -<ipxact:file>
      <ipxact:name>../INTERFACE/initiator_transmitter.v</ipxact:name>
      <ipxact:fileType>verilogSource</ipxact:fileType>
      <ipxact:logicalName>initiator_transmitterlib</ipxact:logicalName>
    </ipxact:file>
  </ipxact:fileSet>
</ipxact:fileSets>
</ipxact:component>
```

- Each port element has a name
 - Typically, the value of the name element is equal to the HDL port name.
- Port can be wire, transactional, or structured element
 - HDL port, a transaction-level port, composition (SystemVerilog interfaces)

- Multiple file sets can be described using the container element fileSet.
- Each fileSet element has a name to identify it and a list of files.
- Each file has a name, type, and logical name (library in which it is compiled)

IP-XACT: Component with Design and Design Configuration



- Contains a `designInstantiation` and a `designConfigurationInstantiation` in view
- VLNV specified for them and contained in separate files

```
module transmitter_is_initiator;
  wire u_initiator_transmitter_sck_sig;
  wire u_initiator_transmitter_ws_sig;
  wire u_initiator_transmitter_sd_sig;

  initiator_transmitter #(
    .my_param (1)
  )
  u_initiator_transmitter (
    .sck( u_initiator_transmitter_sck_sig ),
    .ws( u_initiator_transmitter_ws_sig ),
    .sd( u_initiator_transmitter_sd_sig ) );

  target_receiver u_target_receiver (
    .sck( u_initiator_transmitter_sck_sig ),
    .ws( u_initiator_transmitter_ws_sig ),
    .sd( u_initiator_transmitter_sd_sig ) );

endmodule
```

```
<!--ipxact:component xsi:schemaLocation="http://www.accellera.org/XMLSchema/IPXACT/1685-2022 http://www.accellera.org/XMLScher
<ipxact:vendor>accellera.org</ipxact:vendor>
<ipxact:library>i2s</ipxact:library>
<ipxact:name>transmitter_is_initiator</ipxact:name>
<ipxact:version>1.0</ipxact:version>
<!--ipxact:model>
<!--ipxact:views>
<!--ipxact:view>
<ipxact:name>rtl</ipxact:name>
<ipxact:componentInstantiationRef>hdl-rtl</ipxact:componentInstantiationRef>
<ipxact:designInstantiationRef>hdl-rtl_design</ipxact:designInstantiationRef>
<ipxact:designConfigurationInstantiationRef>hdl-rtl_design_configuration </ipxact:designConfigurationInstantiationRef>
<!--ipxact:view>
</ipxact:views>
<!--ipxact:instantiations>
<!--ipxact:componentInstantiation>
<ipxact:name>hdl-rtl</ipxact:name>
<ipxact:language>verilog</ipxact:language>
<ipxact:libraryName>transmitter_is_initiatorlib</ipxact:libraryName>
<ipxact:moduleName>transmitter_is_initiator</ipxact:moduleName>
<!--ipxact:fileSetRef>
<ipxact:localName>fs-rtl</ipxact:localName>
<!--ipxact:fileSetRef>
<!--ipxact:componentInstantiation>
<!--ipxact:designInstantiation>
<ipxact:name>hdl-rtl_design</ipxact:name>
<ipxact:designRef vendor="accellera.org" library="i2s" name="transmitter_is_initiator_rtl" version="1.0"/>
<!--ipxact:designInstantiation>
<!--ipxact:designConfigurationInstantiation>
<ipxact:name>hdl-rtl_design_configuration</ipxact:name>
<ipxact:designConfigurationRef vendor="accellera.org" library="i2s" name="transmitter_is_initiator_rtl_cfg" version="1.0"/>
<!--ipxact:designConfigurationInstantiation>
</ipxact:instantiations>
<!--ipxact:model>
<!--ipxact:fileSets>
<!--ipxact:fileSet>
<ipxact:name>fs-rtl</ipxact:name>
<!--ipxact:file>
<ipxact:name>./RTL/transmitter_is_initiator_rtl.v</ipxact:name>
<ipxact:fileType>verilogSource</ipxact:fileType>
<ipxact:logicalName>transmitter_is_initiatorlib</ipxact:logicalName>
<!--ipxact:file>
<!--ipxact:fileSet>
</ipxact:fileSets>
</ipxact:component>
```

IP-XACT: Design

```
<ipxact:design xsi:schemaLocation="http://www.accellera.org/XMLSchema/IPXACT/1685-2022 http://www.accellera.org/XMLSchema/IPXACT/1685-2022/index.xsd">
```

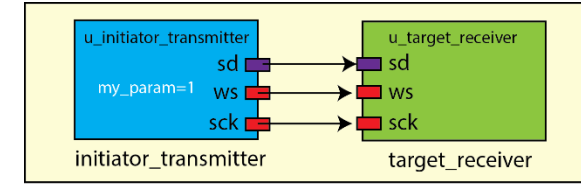
```
<ipxact:vendor>accellera.org</ipxact:vendor>
<ipxact:library>i2s</ipxact:library>
<ipxact:name>transmitter_is_initiator_rtl</ipxact:name>
<ipxact:version>1.0</ipxact:version>
```

VLNV

```
<ipxact:componentInstances>
  <ipxact:componentInstance>
    <ipxact:instanceName>u_initiator_transmitter</ipxact:instanceName>
    <ipxact:componentRef vendor="accellera.org" library="i2s" name="initiator_transmitter" version="1.0"/>
  </ipxact:componentInstance>
  <ipxact:componentInstance>
    <ipxact:instanceName>u_target_receiver</ipxact:instanceName>
    <ipxact:componentRef vendor="accellera.org" library="i2s" name="target_receiver" version="1.0"/>
  </ipxact:componentInstance>
</ipxact:componentInstances>
```

```
<ipxact:adHocConnections>
  <ipxact:adHocConnection>
    <ipxact:name>u_initiator_transmitter_sck_u_target_receiver_sck</ipxact:name>
    <ipxact:portReferences>
      <ipxact:internalPortReference componentInstanceRef="u_initiator_transmitter" portRef="sck"/>
      <ipxact:internalPortReference componentInstanceRef="u_target_receiver" portRef="sck"/>
    </ipxact:portReferences>
  </ipxact:adHocConnection>
  <ipxact:adHocConnection>
    <ipxact:name>u_initiator_transmitter_ws_u_target_receiver_ws</ipxact:name>
    <ipxact:portReferences>
      <ipxact:internalPortReference componentInstanceRef="u_initiator_transmitter" portRef="ws"/>
      <ipxact:internalPortReference componentInstanceRef="u_target_receiver" portRef="ws"/>
    </ipxact:portReferences>
  </ipxact:adHocConnection>
  <ipxact:adHocConnection>
    <ipxact:name>u_initiator_transmitter_sd_u_target_receiver_sd</ipxact:name>
    <ipxact:portReferences>
      <ipxact:internalPortReference componentInstanceRef="u_initiator_transmitter" portRef="sd"/>
      <ipxact:internalPortReference componentInstanceRef="u_target_receiver" portRef="sd"/>
    </ipxact:portReferences>
  </ipxact:adHocConnection>
</ipxact:adHocConnections>
```

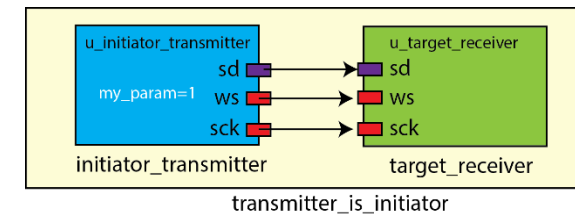
```
</ipxact:design>
```



transmitter_is_initiator

- Contains a **componentInstances**
 - **instanceName** matches with the HDL instance name
 - **componentRef** point to VLNV
-
- **adHocConnections** is a container element for all ad hoc connections in the design
 - **adHocConnection** has a name **internalPortReference** (with references to component and ports)

IP-XACT: Design Configuration



```
<ipxact:designConfiguration xsi:schemaLocation="http://www.accellera.org/XMLSchema/IPXACT/1685-2022 http://www.accellera.org/XMLSchema/IPXACT/1685-2022/index.xsd">
```

```
<ipxact:vendor>accellera.org</ipxact:vendor>
<ipxact:library>i2s</ipxact:library>
<ipxact:name>transmitter_is_initiator_rtl_cfg</ipxact:name>
<ipxact:version>1.0</ipxact:version>
```

VLNV

```
<ipxact:designRef vendor="accellera.org" library="i2s" name="transmitter_is_initiator_rtl" version="1.0"/>
```

```
<ipxact:viewConfiguration>
```

```
<ipxact:instanceName>u_initiator_transmitter</ipxact:instanceName>
```

```
<ipxact:view viewRef="interface">
```

```
<ipxact:configurableElementValues>
```

```
<ipxact:configurableElementValue referenceId="my_param">1</ipxact:configurableElementValue>
```

```
</ipxact:configurableElementValues>
```

```
</ipxact:view>
```

```
</ipxact:viewConfiguration>
```

```
<ipxact:viewConfiguration>
```

```
<ipxact:instanceName>u_target_receiver</ipxact:instanceName>
```

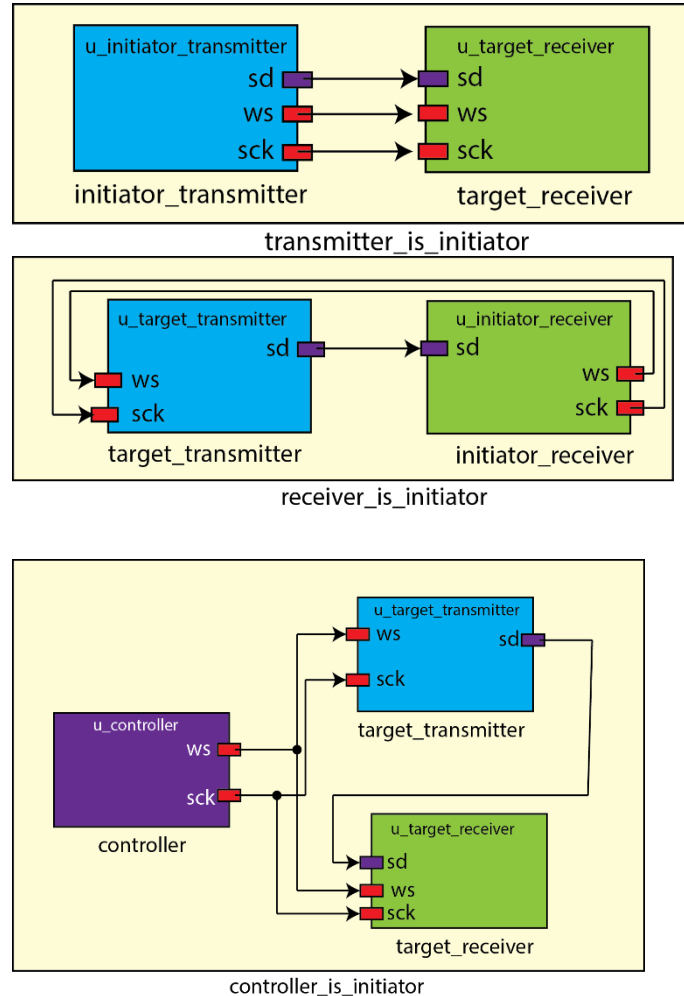
```
<ipxact:view viewRef="interface"/>
```

```
</ipxact:viewConfiguration>
```

```
</ipxact:designConfiguration>
```

- Design configuration describes the configuration of a design in terms of **view** configurations for component instances
- The **viewRef** references a view by its name that exists in the **component** of the referenced **componentInstance**
- Parameter value is defined

IP-XACT Summary: Component, Design, and Design Configuration

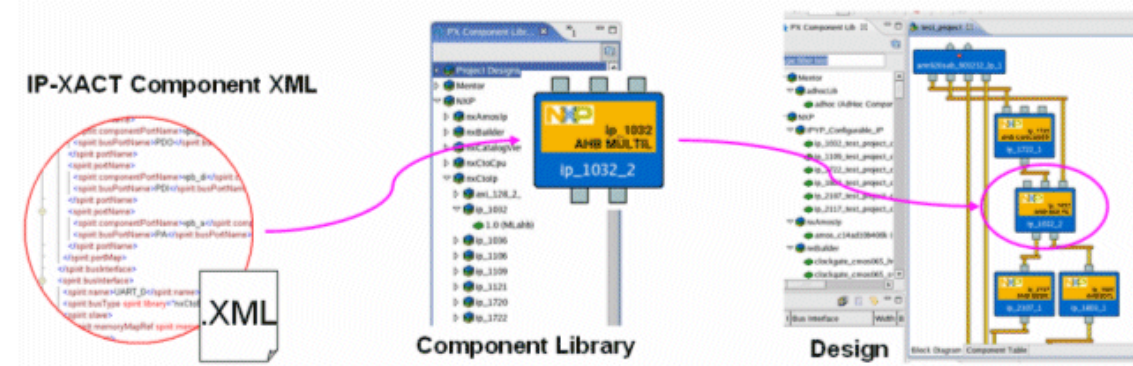


- Module declarations described as views in the IP-XACT **components** `transmitter_is_initiator`, `receiver_is_initiator`, and `controller_is_initiator`
- The internal structure of these three modules is described in three IP-XACT **design**
 - Instantiates components and connect them
- **designConfiguration** for that design describes the views for those component instances (module name can be derived for each component instance)
- **design** and the **designConfiguration** can be referenced from the **view** in the **component** `transmitter_is_initiator`.
- **view** can be used to generate the module `transmitter_is_initiator` in Verilog

IP Assembly

System-on-Chip (SoC) Design Methodology

- Automatic tools used for assembling IPs
 - coreAssembler from Synopsys
 - Kactus2
- Managing connections and parameters are the main challenges
- Generator tools for RTL and verification environment generation



SoC Design using Open-source Tools

Creating IPXACT models

- We can use opensource tool **verilog2ipxact**: <https://www.edautils.com/verilog2ipxact.html>

```
module initiator_transmitter(sck, ws, sd);
    output wire sck;
    output wire ws;
    output wire sd;
endmodule
```

```
module target_receiver(sck, ws, sd);
    input wire sck;
    input wire ws;
    input wire sd;
endmodule
```

```
SNEH2023:~/ipxact/demo_ip_generation> $EDAUTILS_ROOT/bin/verilog2ipxact -in initiator_transmitter.v -out initiator_transmitter.xml -top initiator_transmitter -v2014
verilog2ipxact: verilog2ipxact
verilog2ipxact: Version: 8.5.0 Released on 11 Feb 2024
verilog2ipxact: Support info: help@edautils.com
verilog2ipxact:
```

```
verilog2ipxact: Note: 51921: Found 0 error(s) and 0 warning(s) ...
verilog2ipxact: Note: 51922: Thank you for using this utility
verilog2ipxact: Note: 51923: Send your suggestion/feedback to help@edautils.com
verilog2ipxact: Note: 51924: Exiting ...
```

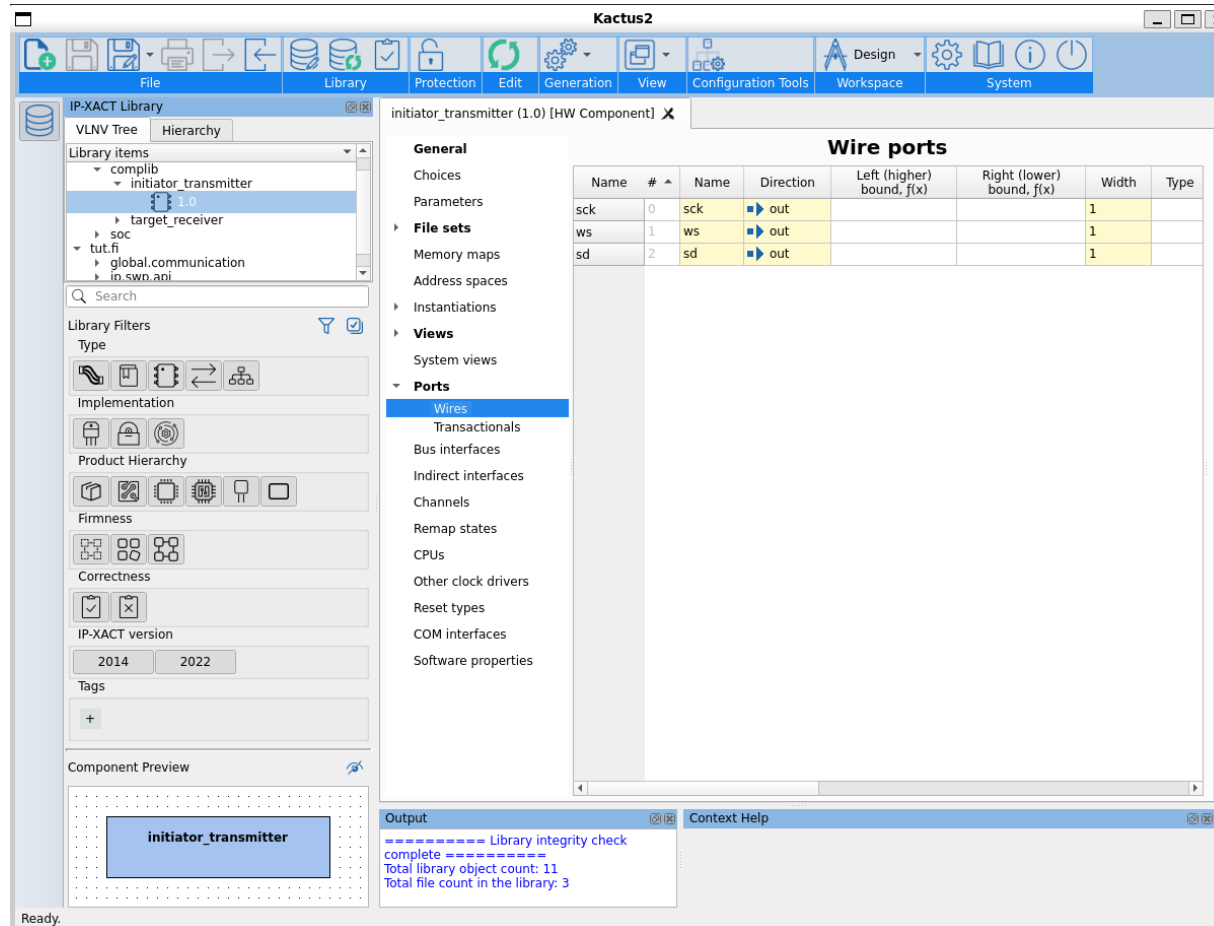
```
SNEH2023:~/ipxact/demo_ip_generation>
```

```
-rw-r--r-- 1 sneh sneh 5055 Sep 18 20:23 verilog2ipxact.log
-rw-r--r-- 1 sneh sneh 2114 Sep 18 20:23 initiator_transmitter.xml
SNEH2023:~/ipxact/demo_ip_generation>
```

```
SNEH2023:~/ipxact/demo_ip_generation> $EDAUTILS_ROOT/bin/verilog2ipxact -in target_receiver.v -out target_receiver.xml -top target_receiver -v2014
verilog2ipxact: verilog2ipxact
verilog2ipxact: Version: 8.5.0 Released on 11 Feb 2024
```

IP Assembly

- We can use opensource tool kactus2: <https://github.com/kactus2/kactus2dev>



High-level Synthesis

High-level Synthesis (HLS): Introduction

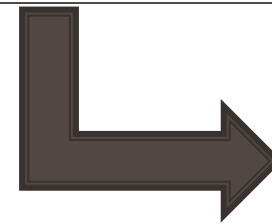
- Also called Behavioral Synthesis
- Converts a design described in algorithmic form (not timed) to an equivalent design in RTL
 - RTL is timed and satisfy the constraints specified by the user.

- **Inputs:**
 - Design/Algorithm usually in SystemC, C++, MATLAB
 - Constraints: resource utilization, latency, delay, power, connectivity
 - Libraries: attributes of resources, protocols
- **Output:**
 - RTL usually in Verilog/VHDL

Motivation

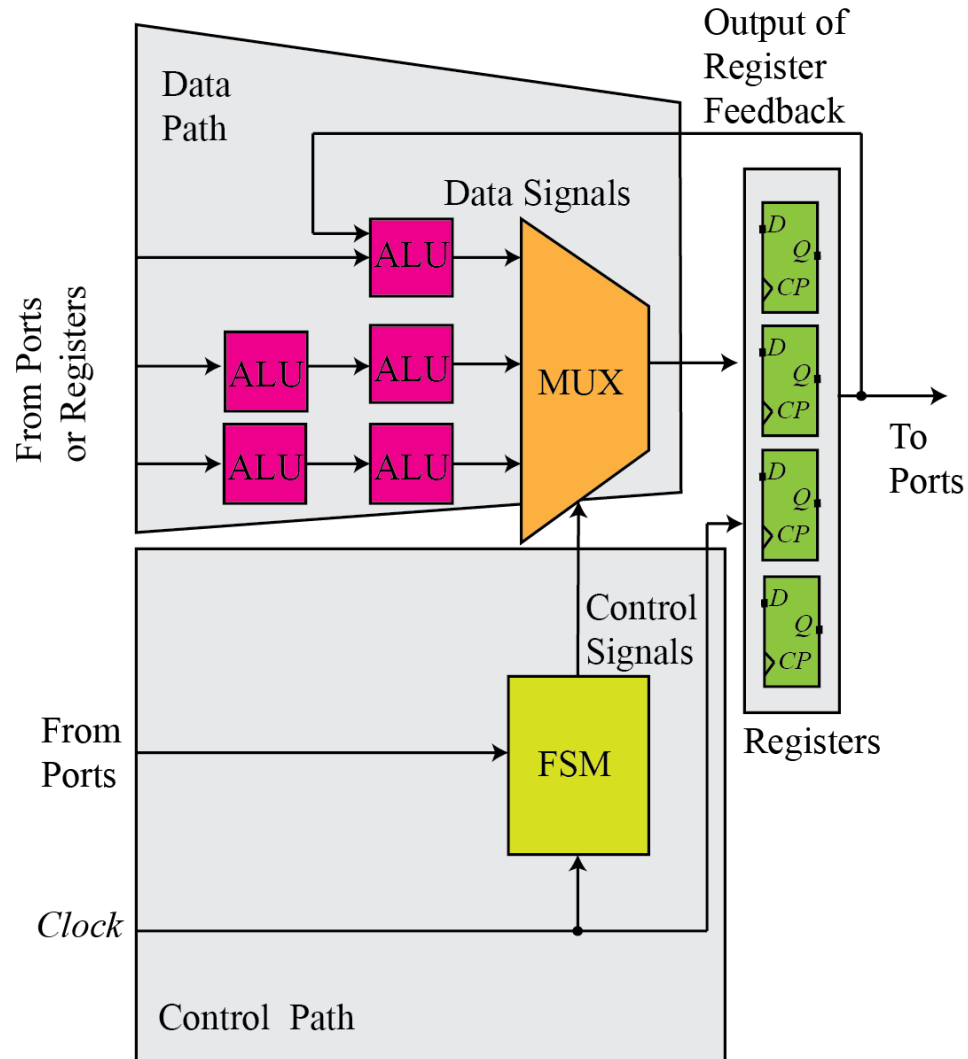
- HLS allows automatic exploration of different possible implementations: more exhaustive than handwritten RTL
 - Especially exploring micro-architectures (pipelining, parallelism, etc.)
- Reduces design effort
- Reduces chance of introducing errors

High-level Synthesis



Transformations

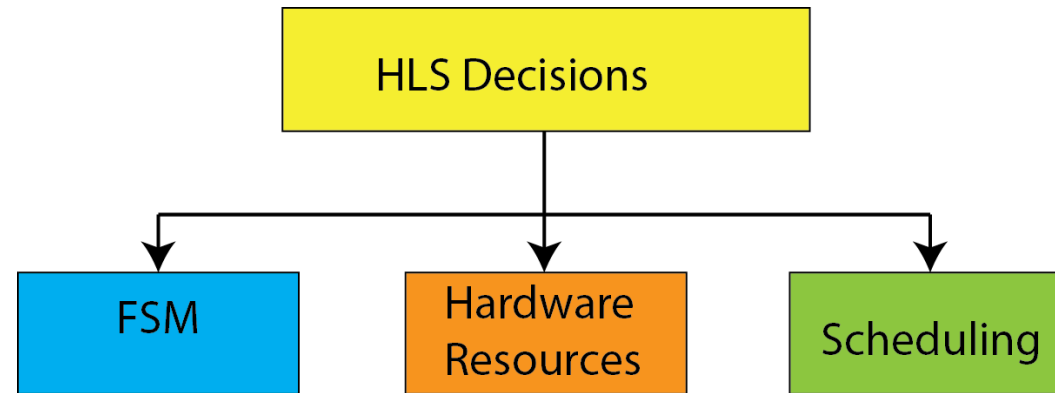
HLS: General Structure of RTL



RTL: Register Transfer Level

- Modelling of circuit as flow of data (signal) between *registers*
 - RTL can also be referred to as “data flow” description
-
- FSM generates control signals
 - MUX passes the data based on *control signals*
 - Computation is done on the data path

HLS: Design Decisions Taken (1)



HLS: Design Decisions Taken (FSM)

Decisions related to FSM:

- FSMs are not explicitly specified in the algorithm
 - RTL specifies a set of interacting FSMs
- HLS need to decide the structure of the FSMs and the interactions between them
- Interaction among FSMs determines concurrency among hardware components (each hardware component is associated with a single FSM)

HLS decides:

- Which part of algorithm is to be captured using single component (FSM)?
- How these components (FSMs) interact?
- For each FSM, the number of states and their transitions

- In practice, tools use structure of the given algorithm to infer components
 - We can use SystemC features such as concurrent threads to model individual components
 - We can put separate components in different functions (C, C++)
- Interactions can be inferred from the given model:
 - Handshaking and TLM can model interactions which is explicit
 - User-defined inputs directly to the tool
- HLS decides the internal structure of each FSM based on various analysis

HLS: Design Decisions Taken (2)

Decide on Hardware resources

- Computation: various flavors
- Storage: wire, registers, memories

Scheduling

- Determines which computation to do in which states of the FSMs
- Difference in clock cycles at which computation element produces a result and the clock cycle at which the result is consumed
 - HLS tool has a flexibility to schedule a computation

Challenge

- All three decisions are inter-dependent
- Making decisions can be difficult

Optimization

- Simplifies given algorithm
- Discovers opportunities of parallelism

References

- S. Saurabh, “Introduction to VLSI Design Flow”. Cambridge: *Cambridge University Press*, 2023.
- <https://www.edn.com/system-on-chip-soc-design-is-all-about-ip-management/>
- <https://accelera.org/downloads/standards/ip-xact>